

# MEMORY MANAGEMENT SIMULATOR

## Project Proposal



**Supervisor**

Sir Mehtab Mushtaq

**Submitted by**

M. Siraj ud din  
(Roll No:74)

Farhan Fiaz  
(Roll No:79)

Owais Shezad  
(Roll No:83)

BSCS 3<sup>rd</sup> (B)

Session 2023-2027

Department of CS & IT

Faculty of Computig & Engineering

University of Kotli Azad Kashmir

Submission date

[30-01-26]

## TABLE OF CONTENTS

|                                                      |   |
|------------------------------------------------------|---|
| <b>1. INTRODUCTION</b>                               | 4 |
| 1.1. OVERVIEW OF MEMORYMANAGEMENT                    | 4 |
| 1.2. IMPORTANCE OF MEMORY MANAGEMENT                 | 4 |
| 1.3. PURPOSE OF THE PROJECT                          | 4 |
| 1.3.1.General Objectives                             | 4 |
| 1.3.2. Specific Objectives                           | 4 |
| <b>2. BACKGROUND AND LITERATURE REVIEW</b>           | 4 |
| 2.1. MEMORY ALLOCATION TECHNIQUES                    | 4 |
| 2.2. CONTIGUOUS MEMORY ALLOCATION                    | 4 |
| 2.2.1. First Fit Algorithm                           | 5 |
| 2.2.2. Next Fit Algorithm                            | 5 |
| 2.2.3. Best Fit Algorithm                            | 5 |
| 2.2.4. Worst Fit Algorithm                           | 5 |
| <b>3.PAGING AND SEGMENTATION</b>                     | 5 |
| 3.1. PAGING TECHNIQUE                                | 5 |
| 3.1.1. Advantages of Paging                          | 5 |
| 3.2. SEGMENTATION TECHNIQUE                          | 5 |
| 3.2.1. Advantages of Segmentation                    | 5 |
| 3.2.2. Difference Between Paging and Segmentation    | 5 |
| <b>4. INPUT AND OUTPUT REQUIREMENTS</b>              | 6 |
| 4.1. INPUT REQUIREMENTS                              | 6 |
| 4.2. OUTPUT REQUIREMENTS                             | 6 |
| <b>5. FUNCTIONAL AND NON FUNCTIONAL REQUIREMENTS</b> | 6 |
| 5.1. FUNCTIONAL REQUIREMENTS                         | 6 |
| 5.2. NON FUNCTIONAL REQUIREMENTS                     | 6 |
| <b>6. ASSEPTIONS AND CONSTRAINTS</b>                 | 6 |
| <b>7.SYSTEM DIAGRAM AND OVERVIEW</b>                 | 7 |
| <b>8. PROJECT SCOPE AND LIMITATIONS</b>              | 8 |
| <b>9. TOOLS AND TECHNOLOGIES</b>                     | 8 |

|                                                |          |
|------------------------------------------------|----------|
| 9.1. SOFTWARE TOOLS.....                       | 8        |
| 9.2. PLATEFORMS.....                           | 8        |
| <b>10.TEAM ROLES AND RESPONSIBILITIES.....</b> | <b>8</b> |
| <b>11. CONCLUSION.....</b>                     | <b>8</b> |
| <b>12. REFERENCE.....</b>                      | <b>8</b> |

# **1.INTRODUCTION**

## **1.1. OVERVIEW OF MEMORY MANAGEMENT**

Memory management is one of the most critical functions of an operating system. It is responsible for managing the allocation, utilization, and de-allocation of main memory among running processes. Efficient memory management ensures better system performance, enables multitasking, and prevents issues such as memory leaks and fragmentation.

## **1.2. IMPORTANCE OF MEMORY MANAGEMENT**

Modern operating systems must support multiple users and applications simultaneously. Without proper memory management, system performance degrades, leading to crashes or inefficient utilization of hardware resources.

## **1.3. PUPOSE OF THE PROJECT**

The purpose of this project is to design and document a Memory Management Simulator that visually demonstrates how different memory allocation techniques work.

### **1.3.1.General Objectives**

To design a visual simulator that improves understanding of memory management concepts in operating systems.

### **1.3.2. Specific Objectives**

- ♦ Implement First Fit, Next Fit, Best Fit, and Worst Fit algorithms
- ♦ Simulate paging and segmentation
- ♦ Visualize memory allocation and fragmentation

# **2. BACKGROUND AND LITERATURE REVIEW**

## **2.1. MEMORY ALLOCATION TECHNIQUES**

Memory allocation techniques are used by operating systems to assign memory to processes. These techniques can be categorized into contiguous and non-contiguous allocation methods.

## **2.2. CONTIGUOUS MEMORY ALLOCATION**

In contiguous memory allocation, each process is assigned a single continuous block of memory.

### 2.2.1. First Fit Algorithm

First Fit allocates the first available free memory block that is large enough to accommodate the process.

Advantages: Simple and fast

### 2.2.2. Next Fit Algorithm

Next Fit is similar to First Fit, but it continues searching from the last allocated position instead of starting from the beginning of memory.

### 2.2.3. Best Fit Algorithm

Best Fit allocates the smallest free block that satisfies the process's memory requirement.

Advantage: Reduces wasted memory

### 2.2.4. Worst Fit Algorithm

Worst Fit allocates the largest available memory block to a process, aiming to reduce the creation of small fragments.

## 3. PAGING AND SEGMENTATION

### 3.1. PAGING TECHNIQUE

Paging divides physical memory into fixed-size frames and logical memory into pages. Each page is mapped to a frame using a page table.

#### 3.1.1. Advantages of Paging

- Eliminates external fragmentation
- Efficient memory utilization

### 3.2. SEGMENTATION TECHNIQUE

Segmentation divides a program into logical segments such as code, data, and stack.

#### 3.2.1. Advantages of Segmentation

- Logical memory organization
- Supports protection and sharing

#### 3.2.2. Difference Between Paging and Segmentation

Paging is fixed-size and hardware-based, while segmentation is variable-size and logical.

## **4.INPUT AND OUTPUT REQUIREMENTS**

### **4.1. INPUT REQUIREMENTS**

- The total size of main memory to be used for simulation.
- Process identifier (Process ID or process name).
- User selection to allocate memory for a process and de-allocate a process from memory.
- Selection of memory allocation techniques (eg. First fit ).

### **4.2. OUTPUT REQUIREMENTS**

- Display allocated memory blocks with processes details.
- Display of free memory blocks after allocation and de-allocation.
- Overall memory status in a clear and user friendly format.
- Update memory layout after each operation.

## **5. FUNCTIONAL AND NON FUNCTIONAL REQUIREMENTS**

### **5.1. FUNCTIONAL REQUIREMENTS**

- ◆ Allow users to input multiple processes and their memory requirements.
- ◆ Implement memory allocation algorithms: First Fit, Next Fit, Best Fit, and Worst Fit.
- ◆ Simulate paging and segmentation, showing memory as frames or logical segments.
- ◆ Allow memory deallocation when a process terminates or no longer needs memory.

### **5.2. NON FUNCTIONAL REQUIREMENTS**

- ◆ User-friendly and intuitive interface for easy navigation.
- ◆ Compatible with Windows and Linux platforms.
- ◆ Reliable and accurate visualization of memory management processes.
- ◆ Lightweight and implemented using open-source tools (Python and Tkinter).

## **6.ASSEMPTIONS AND CONSTRAINTS**

- ◆ All user inputs are assumed to be valid and positive.
- ◆ The total size of memory is fixed during simulation.
- ◆ The simulator supports a limited number of processes.
- ◆ Only one memory allocation algorithm used at a time.
- ◆ is developed for educational purpose only.

## 7. SYSTEM DIAGRAM AND OVERVIEW

This project implements a Memory Management Simulator that demonstrates how an operating system manages main memory. The simulator performs core memory operations such as allocation, de-allocation, paging, and segmentation using classical memory allocation algorithms including First Fit, Best Fit, Next Fit, and Worst Fit. By visualizing memory blocks and process placement, the project helps users understand how different algorithms affect memory utilization and fragmentation. The simulator is inspired by existing tools such as **MemSim**, and related screenshots are included for reference and comparison.

**First Fit Allocation** Home First Fit Best Fit Next Fit Worst Fit Paging Segmentation

Memory Blocks (comma separated sizes):  
820, 794, 398, 980, 880

Processes (comma separated sizes):  
135, 75, 160, 462, 449

Start Allocation Random Values

Visual representation of memory layout:

| Process | Size   | Allocated Block |
|---------|--------|-----------------|
| P1      | 135 KB | Block 1         |
| P2      | 75 KB  | Block 2         |
| P3      | 160 KB | Block 3         |
| P4      | 462 KB | Block 5         |
| P5      | 449 KB | Block 4         |

Fragment 1 KB

Fragment 332 KB

Block 3 398 KB

Block 4 980 KB

Block 5 880 KB

**First Fit Allocation** Home First Fit Best Fit Next Fit Worst Fit Paging Segmentation

| PROCESS | SIZE   | ALLOCATED BLOCK |
|---------|--------|-----------------|
| P1      | 135 KB | Block 1         |
| P2      | 75 KB  | Block 2         |
| P3      | 160 KB | Block 3         |
| P4      | 462 KB | Block 5         |
| P5      | 449 KB | Block 4         |

Allocation done. Total external fragmentation: 2591 KB

## **8. PROJECT SCOPE AND LIMITATIONS**

The project focuses on educational simulation. Real OS-level memory management and hardware interaction are beyond the scope.

## **9. TOOLS AND TECHNOLOGIES**

### **9.1. SOFTWARE TOOLS**

- ♦ Python Programming Language
- ♦ Tkinter GUI Library
- ♦ Visual Studio Code

### **9.2. PLATFORMS**

- ♦ Windows
- ♦ Linux

## **10. TEAM ROLES AND RESPONSIBILITIES**

All the team members contributed equally to this project. The team collaboratively gathered and analyzed the requirements, discussed the system objectives and finalized the functional and non-functional requirements. Each member participated in brainstorming, documentation writing and review to ensure clarity, accuracy and completeness of the requirement specification.

## **11. CONCLUSION**

The Memory Management Simulator provides an effective platform for understanding complex memory management concepts through visualization and interaction.

## **12. REFERENCE**

1. GeeksforGeeks. Memory management in operating system.
2. Python Software Foundation. Python documentation. <https://docs.python.org>
3. tkinter GUI programming Documentation. Python Tkinter library. <https://docs.python.org/3/library/tkinter.html>
4. Chatgpt. (Assistance in rewriting and refining content for clarity and structure.)
5. Hindi tutorials (For Algorithms)